Module 4 Assessment Project

FSM Design, Sequential Logic & Modular SystemVerilog

Before you write any RTL:

Mentees must first complete the design and block diagram for the chosen project track including the FSM state diagram, module boundaries, and top-level interface before starting implementation. This design should be reviewed (self-checked against the specification, or checked with your mentor) before you open your editor and start writing SystemVerilog. Skipping this step is the most common cause of wasted debugging time in this assessment.

1. Overview

This assessment consolidates what you've learned about finite state machine (FSM) design, sequential logic, and modular SystemVerilog coding. You will design, implement, simulate, and verify a small but complete digital system — starting from a written specification and ending with a working, testable RTL design.

Key facts about the assessment:

- Scoped to be completed within 2–3 working days.
- Built entirely from concepts you already know: FSMs, counters, comparators, and basic arithmetic.
- No new SystemVerilog constructs are required — the challenge is structuring your design cleanly and verifying it thoroughly.

1.1 Learning Objectives

- Design a multi-state FSM (Moore or Mealy style) directly from a written specification, including states, inputs, outputs, and transition conditions.
- Implement supporting datapath logic such as counters, comparators, shift/sequence registers, and simple BCD arithmetic.
- Practice modular design: keep your FSM (control path) and datapath logic (registers, comparators, arithmetic) in separate, clearly connected modules.
- Write a self-checking SystemVerilog testbench that automatically flags pass/fail conditions, rather than requiring manual waveform inspection for every case.
- Use waveform analysis (QuestaSim / GTKWave) to debug timing and logic issues systematically, rather than by trial and error.
- Communicate a design clearly through an FSM diagram and a short written justification of your design decisions.

1.2 Tools

- QuestaSim (or an equivalent SystemVerilog simulator provided by your lab) for compilation and simulation.
- GTKWave
- EDA Playground

1.3 Deliverables

Deliverables fall into two phases. Complete the design phase in full before starting the implementation phase — see the note at the top of this document.

Design phase (complete first)

- Draft FSM diagram (states, transitions, and conditions) for your chosen track.
- Block diagram showing module boundaries and how the control path (FSM) connects to the datapath (registers, comparators, arithmetic).
- A brief note on Moore vs. Mealy choice, if relevant.

Implementation phase

- RTL source files (.sv) — one file per module, clearly named.
- A self-checking testbench (.sv) covering at least the scenarios listed under "Verification Checklist" for your chosen track.
- Waveform screenshots showing at least one passing run and one deliberately-triggered failure/edge case, each with a caption explaining what it shows.
- A one-page design write-up containing: your final FSM diagram, a short explanation of your Moore vs. Mealy choice (if relevant), and a list of any bugs you encountered and how you resolved them.

1.4 Grading Emphasis

Area	**What is assessed**
Correctness	FSM and datapath behave correctly across all specified scenarios, including edge cases.
Design structure	Clear separation of control (FSM) and datapath logic; sensible, consistent naming.
Verification	Testbench is self-checking (uses assertions or automatic pass/fail reporting), not just a waveform dump.
Debugging & write-up	Evidence of systematic debugging; write-up clearly explains design decisions and issues encountered.

2. Project Track A — Simon Says Memory Game

The system generates a pseudo-random sequence of LED flashes. The player must repeat the sequence in order by pressing the corresponding buttons. Each round the player completes correctly, the sequence grows by one additional step. The game ends the moment the player presses a wrong button, at which point the final score (number of rounds survived) is displayed.

2.1 Functional Specification

The specification is broken down below by system behavior, in the order it occurs during a game.

Start / Reset

- On reset or a start button press, the system enters IDLE and waits for the player to begin.

Sequence generation and display

- The system generates the next random step and appends it to the current sequence.

- It then re-plays the full sequence from the beginning: LEDs light one at a time, with a visible delay between steps, generated via a clock divider.

Player input

- After the sequence finishes displaying, the system waits for the player to press buttons in the same order.

Round outcomes

- Correct sequence: if every button press matches the corresponding step, the round is won — the level counter increments, and the system loops back to generate and display a longer sequence.
- Incorrect press: if any button press does not match, the system enters GAME_OVER, freezes the score on the display, and waits for a reset/restart signal.

2.2 Day-by-Day Breakdown

Day 1 — Sequence Generation and Display

- Implement a pseudo-random sequence generator. A simple linear-feedback shift register (LFSR), seeded at reset, is sufficient — you do not need cryptographic-quality randomness, just a sequence the player can't predict.
- Implement a clock divider to slow your fast system clock to a human-visible rate (roughly 2–4 steps per second is a good starting point). Make this a parameter so it's easy to tune during testing.
- Implement the SHOW_PATTERN state: on entry, step through the stored sequence from the beginning, driving one LED output at a time for one “tick” of the divided clock each.
- Checkpoint: simulate in isolation and confirm, via waveform, that the correct LED lights at the correct time for a 3–4 step sequence.

Day 2 — Player Input and Comparison

- Implement the WAIT_INPUT state: sample the player's button inputs (debouncing if your target board requires it), and compare each press against the corresponding step in the stored sequence, one step at a time.
- On a correct press: if steps remain in this round, stay in WAIT_INPUT and advance an internal step-index counter. If the player has just matched the final step, transition to LEVEL_UP.
- On an incorrect press: transition immediately to GAME_OVER.
- Checkpoint: write a testbench that drives one fully-correct sequence through to LEVEL_UP, and a second run that deliberately injects one wrong button press partway through, confirming the FSM reaches GAME_OVER at the right step.

Day 3 — Scoring, Level Growth, and Polish

- In LEVEL_UP, increment a level/score counter and append one new random step to the sequence (reusing your Day 1 generator), then loop back to SHOW_PATTERN.
- Drive a 7-segment display (or equivalent) from the score counter, converting binary to the appropriate segment pattern.
- Add a clear win/fail indicator (a dedicated LED or buzzer output) that activates in GAME_OVER.
- Stretch goal, if time allows: add a difficulty setting (switch input) that changes the clock-divider tick rate.

2.3 FSM Diagram

Insert your FSM diagram here (states: IDLE, SHOW_PATTERN, WAIT_INPUT, LEVEL_UP, GAME_OVER, and their transition conditions). This should be drafted during the design phase, before implementation, and finalized once your design is verified.

2.4 Suggested Module Interface

```
module simon_says (
    input  logic       clk,
    input  logic       rst_n,
    input  logic       start_btn,   // begins a new game from IDLE
    input  logic [3:0] btn_in,      // 4 player buttons, one-hot per press
    output logic [3:0] led_out,     // 4 LEDs, one per sequence color/position
    output logic [6:0] seg_score,   // 7-segment score display (active pattern)
    output logic       game_over    // high while in GAME_OVER
);
```

2.5 Verification Checklist

- A short sequence (e.g., 3 steps) is generated and displayed with correct timing.
- A fully correct player response advances to LEVEL_UP and the sequence grows by exactly one step.
- An incorrect response at the first step, a middle step, and the last step of a sequence each correctly trigger GAME_OVER — test all three positions separately, not just one.
- The score display shows the correct value after several successful rounds.
- A reset during any state correctly returns the system to IDLE with the score cleared.

5. Reference Readings

These references are recommended for FSM design theory, SystemVerilog syntax, and verification practice. You are not expected to read them cover-to-cover — use them as targeted references when you get stuck on a specific concept.

Sarah L. Harris and David Money Harris — Digital Design and Computer Architecture (RISC-V Edition)

Chapter 3 covers sequential logic and FSM design (Moore vs. Mealy, state encoding, timing) in the same style used in this course, and the book's SystemVerilog examples are a good syntax reference. Particularly relevant since this text is co-authored by Prof. David Harris.

Pong P. Chu — FPGA Prototyping by SystemVerilog Examples

Contains complete worked examples of small FSM-based systems (traffic-light controllers, simple games, 7-segment display drivers) that are structurally very close to both project tracks in this guide — useful for pattern-matching your module structure and testbench style.

Samir Palnitkar — Verilog HDL: A Guide to Digital Design and Verification

A strong reference for testbench and verification style if you want to go beyond a minimal self-checking testbench — covers structured verification approaches that translate directly to SystemVerilog.

Clifford E. Cummings — “FSM Coding Styles” and related SystemVerilog papers (SNUG proceedings)

Widely used, freely available conference papers covering common FSM coding pitfalls (incomplete sensitivity lists, unintended latches, reset styles) — worth a quick read before you finalize your FSM code. The underlying design guidance still applies directly, even though the papers predate SystemVerilog's cleaner syntax.